

TRABALHO 2

Bitnópolis foi eleita a SmartCity Modelo da América Latina pela 5ª vez na década. Tamanho sucesso é resultado de um grande empenho em aceitar e resolver desafios. O desafio atual é melhorar a fluidez do sistema viário da cidade. Suspeita-se que a cidade contenha regiões com boa fluidez viária entre-cortadas por regiões de baixa fluidez, configurando diferentes “ilhas” viárias. A sua tarefa identificar estas ilhas e sugerir ampliações viárias que melhorem a interconexão destas ilhas.

A cidade possui ruas, avenidas e rotatórias. Também queremos determinar trajetos na cidade. Uma possível consulta seria:

"Qual é o melhor (mais curto, mais rápido) trajeto entre os endereço X e Y?".

O resultado das consultas são textuais e pictóricas. Um exemplo de uma resposta textual poderia ser:

Siga na direção norte na Rua Xxxx até o cruzamento com a Rua Yyyy. Siga na Rua Yyyy na direção sul.....

O resultado pictórico é mostrado no arquivo .svg produzido, evidenciando o caminho a ser percorrido. Note que são apresentados dois caminhos: o mais rápido e o mais curto.

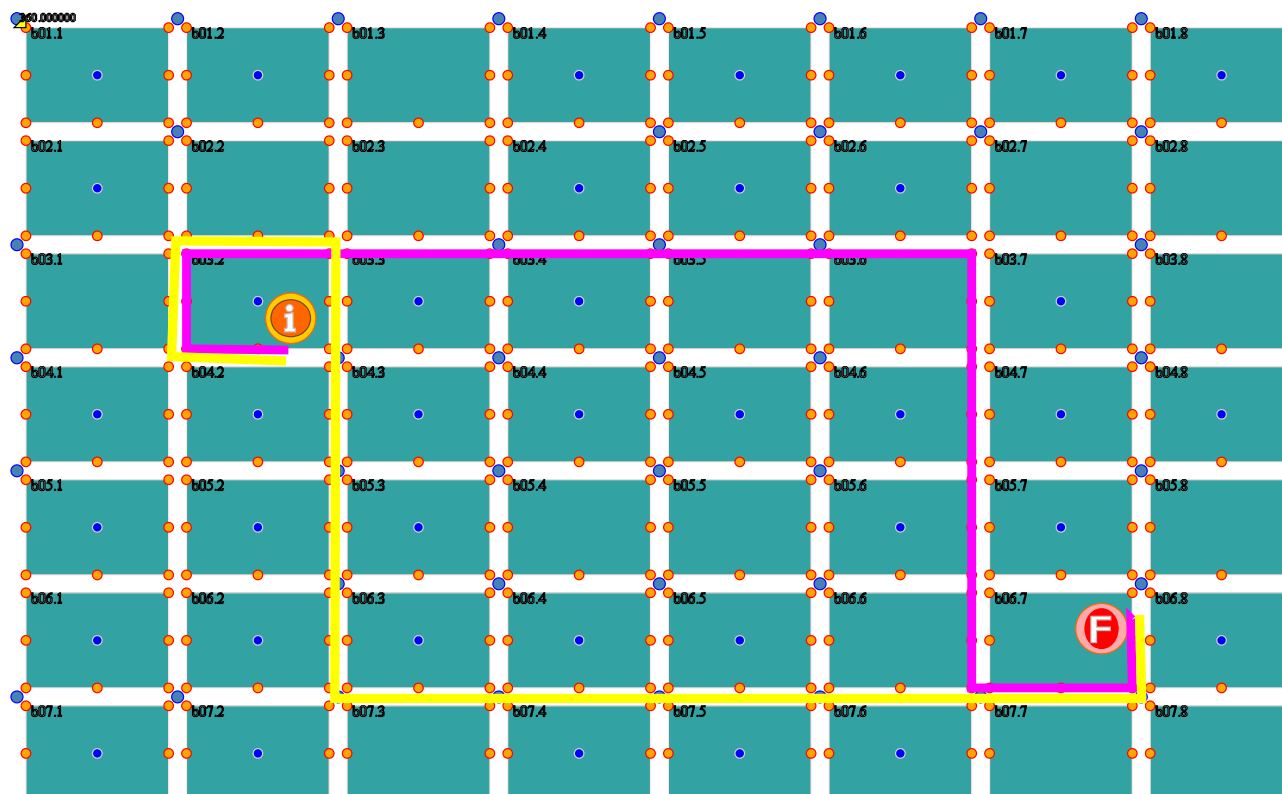


Figura 1: Exemplo de saída de uma consulta. Note o percurso mais rápido e o mais curto.

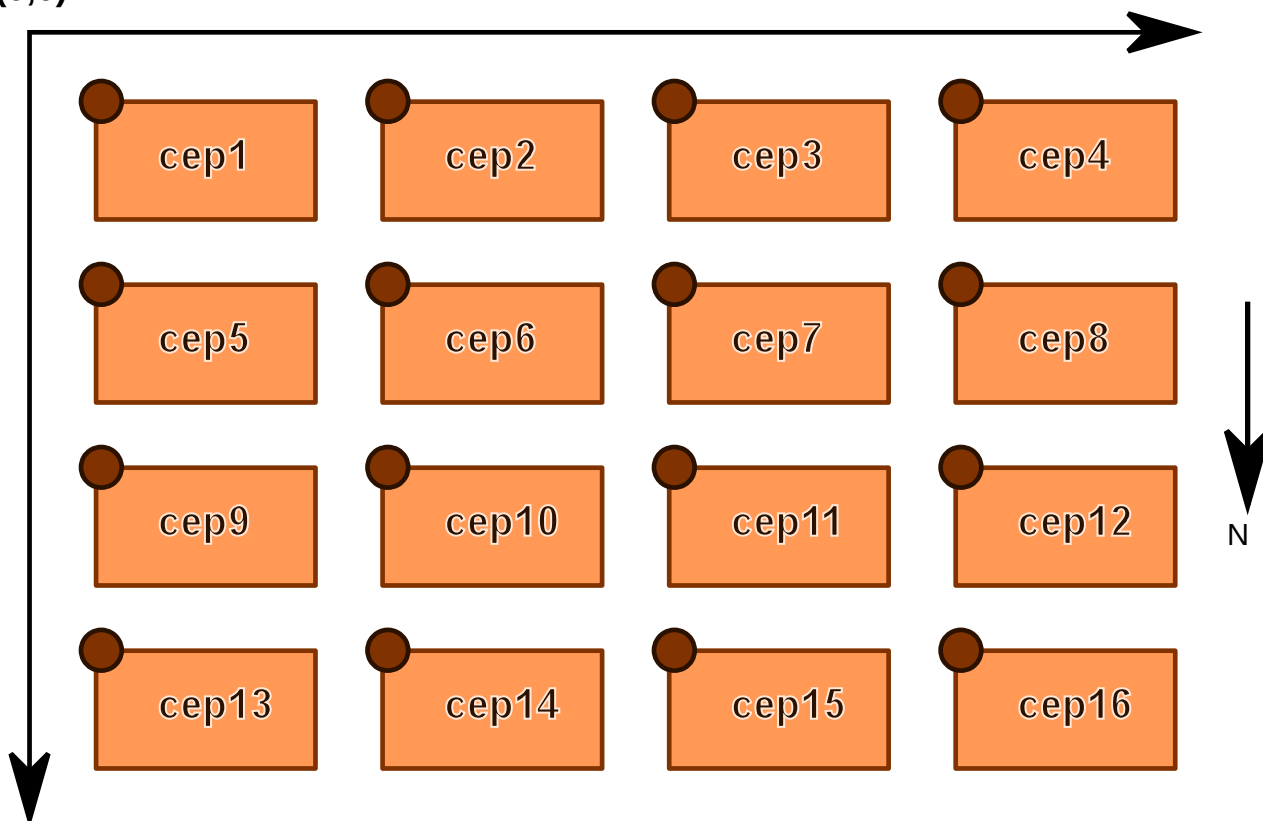
Um sistema que resolve este tipo de problema é denominado **Sistema de Informações**

Geográficas (SIG). Para a nossa finalidade, este é um sistema que contém (não exclusivamente) dados geo-referenciados, isto é, dados com algum atributo de localização espacial (uma coordenada).

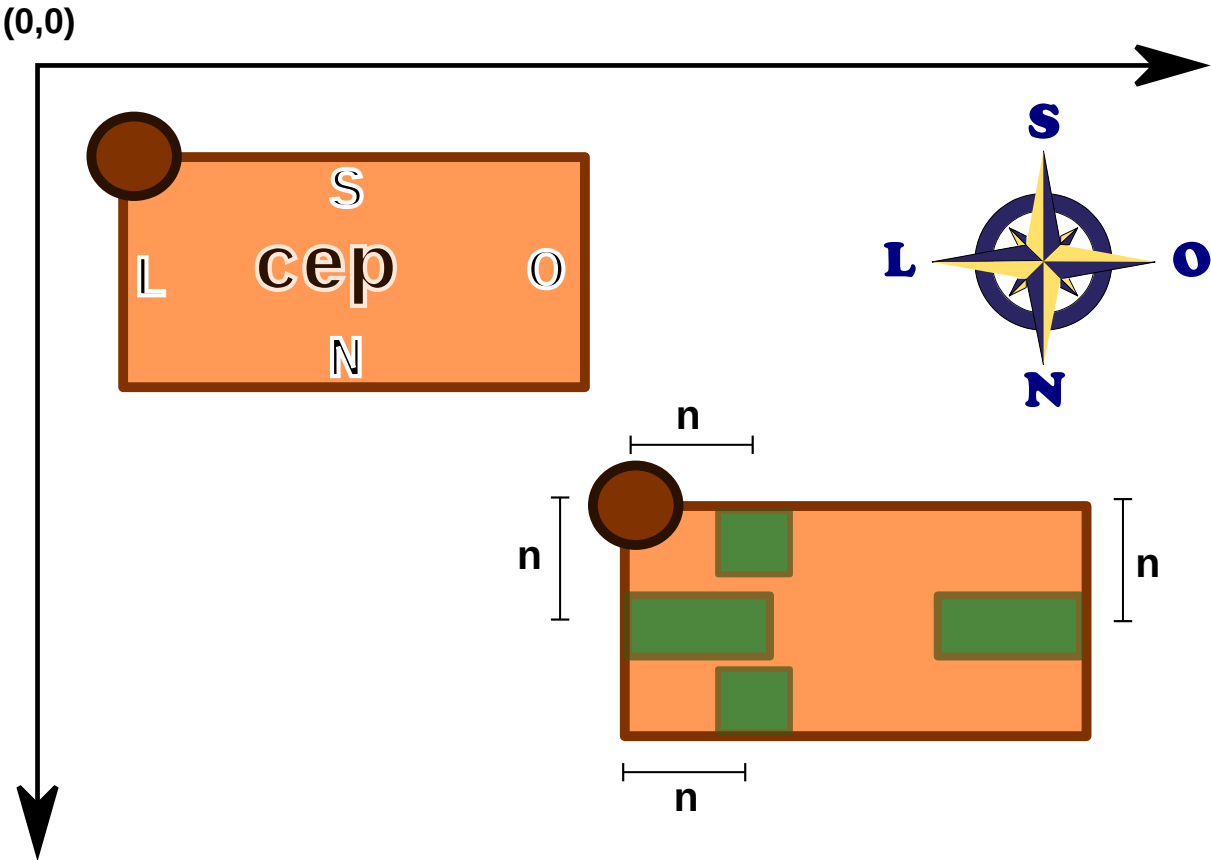
O sistema manipulará o mapa de uma cidade e algumas informações relacionadas.

O mapa de uma cidade é, basicamente, composto por um conjunto de retângulos que representam as quadras.

(0,0)



A cidade exemplificada acima possui 16 quadras. O sistema de endereçamento de Bitnópolis é inspirado no de nossa capital federal. Cada **quadra** possui 4 **faces** (N,S,L,O) e é identificada por um **CEP** alfanumérico. O número de uma casa ou estabelecimento comercial é a **distância** da frente da casa até uma projeção do ponto de ancoragem do retângulo que representa a quadra (veja figura abaixo). Assim, um endereço é da forma CEP/Face/número, por exemplo, cep15/S/45. O ponto de ancoragem do retângulo é o canto sudeste da quadra.



Algumas consultas podem modificar as condições do tráfego. Estas alterações impactam o cálculo do percurso.

A Entrada

Bitnópolis tem quadras e um sistema viário.

Cidade

A cidade é descrita pelo arquivo `.geo`:

comando	parâmetros	
q	cep x y w h	<i>Insera uma quadra (retângulo e cep)</i>
cq	sw cfill cstrk	<i>Cores do preenchimento (cfill) e da borda (cstrk) das quadras, espessura da borda (sw) (a partir deste comando)</i>
Novos comandos do arquivo <code>.geo</code>		

O Sistema Viário

A entrada de dados será feita arquivo-texto (**arquivo.via**) com a descrição do sistema viário da cidade. O sistema viário da cidade é representado por um grafo direcionado.

O Mapa Viário

O mapa viário é um grafo direcionado: os vértices representam os extremos de um segmento de rua e os arcos representam um segmento de rua e indicam o sentido do tráfego.

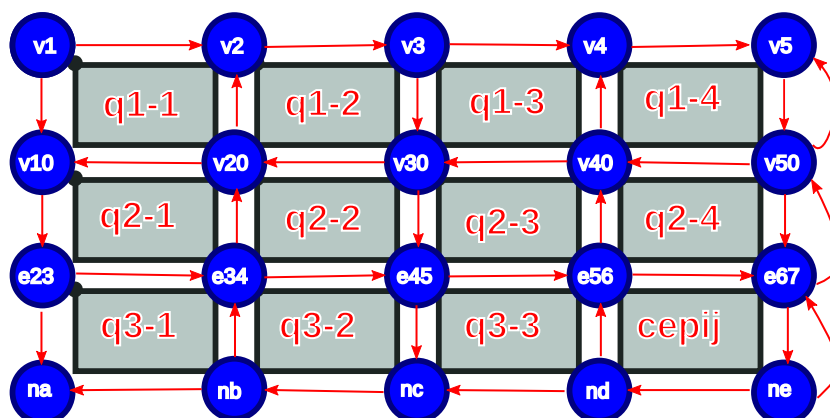


Figura 2: Representação do mapa viário: vértices nas esquinas, arestas indicando o sentido do tráfego.

Os vértices e os arcos possuem alguns atributos:

Atributo	Descrição
Vértices	
id	Um identificador (string)
x,y	Uma coordenada do segmento de rua
Arestas	
nome	Nome da rua a qual pertence o segmento
ldir	Cep da quadra que está do lado direito do segmento de rua
lesq	Idem para o lado esquerdo
cmp	comprimento (em metros) do segmento de rua
vm	Velocidade média (m/s) que os carros trafegam neste segmento de rua

O Arquivo do Mapa Viário

O arquivo do mapa viário possui o seguinte formato (os campos são separados por um espaço em branco):

nv	<i>A primeira linha do arquivo contém um número inteiro (número de vértices do grafo)</i>
v id x y	<i>cria o vértice id posicionado nas coordenadas [x,y]</i>
e i j ldir lesq cmp vm nome	<i>cria a aresta (i,j) e associa as outras informações à aresta. Caso a aresta não possua quadras em algum de seus lados, esta ausência é indicada por um hífen (-)</i>

Abaixo, um exemplo deste arquivo:

```
153
v v1 10.0 10.0
v v2 110.0 10.0
v v3 210.0 10.0
v v4 310.0 10.0
v v5 410.0 10.0
v v6 10.0 70.0
v v7 110.0 70.0
v v8 210.0 70.0
...
e v1 v6 - cep1 70.0 3.5 Rua_Belo_Horizonte
e v7 v8 cep6 cep2 100.0 4.0 Av_Higienopolis
e v8 v7 cep2 cep6 100.0 5.0 Av_10_de_Dezembro
...
mapa-viario1.via
```

Note que a especificação dos vértices sempre precedem as das arestas.

Consultas

As consultas estão descritas na tabela abaixo. Uma das consultas indaga sobre o melhor percurso entre uma origem e um destino. A origem e o destino são referências geográficas. Referências geográficas são obtidas por meio dos comandos iniciados por @ e armazenadas nos registradores R0..R10. A referência geográfica armazenada num registrador pode ser utilizada em outras consultas.

comando	parâmetros	
@o?	reg cep face num	<i>Armazena a posição geográfica do endereço cep/face/num no registrador reg.</i> SVG: linha vertical pontilhada vermelha mostrando a posição do endereço e colocar o número do registrador na outra extremidade do linha (topo da página) TXT: reportar a coordenada relativa ao endereço

mvm	<code>v x y w h</code>	Atualiza a velocidade média das arestas dentro da região (<code>x</code> , <code>y</code> , <code>w</code> , <code>h</code>) para <code>v</code> .
regs	<code>vl</code>	Considera como insuficiente os trechos com velocidade média inferior a <code>vl</code> . Calcula os componentes conexos. TXT : reportar o número de componentes conexos. SVG : Calcular os bounding boxes de cada componente conexo e desenhar os respectivos retângulos com cores diferentes e transparência de 50%.
exp	<code>vl</code>	Calcula a árvore geradora mínima, seleciona apenas as arestas com velocidade média inferior a <code>vl</code> , aumenta em 50% a velocidade média das arestas selecionadas SVG : pintar as arestas selecionadas com linha grossa e vermelha.
p?	<code>reg1 reg2 cc cr</code>	Determina o melhor trajeto entre a origem (<code>reg1</code>) e o destino (<code>reg2</code>). Desenhar os percursos (mais curto, mais rápido) com as cores <code>cc</code> e <code>cr</code> , respectivamente. TXT : descrição textual do percurso. Caso destino inacessível, reportar. SVG : mostrar os percursos pintados e animados. Marcar os extremos do percurso com “placas” com as letras I e F.

Comandos do arquivo .qry

A Figura 3 ilustra uma possível saída do comando `regs`. Esta figura mostra os retângulos envolventes de seus 4 componentes conexos.

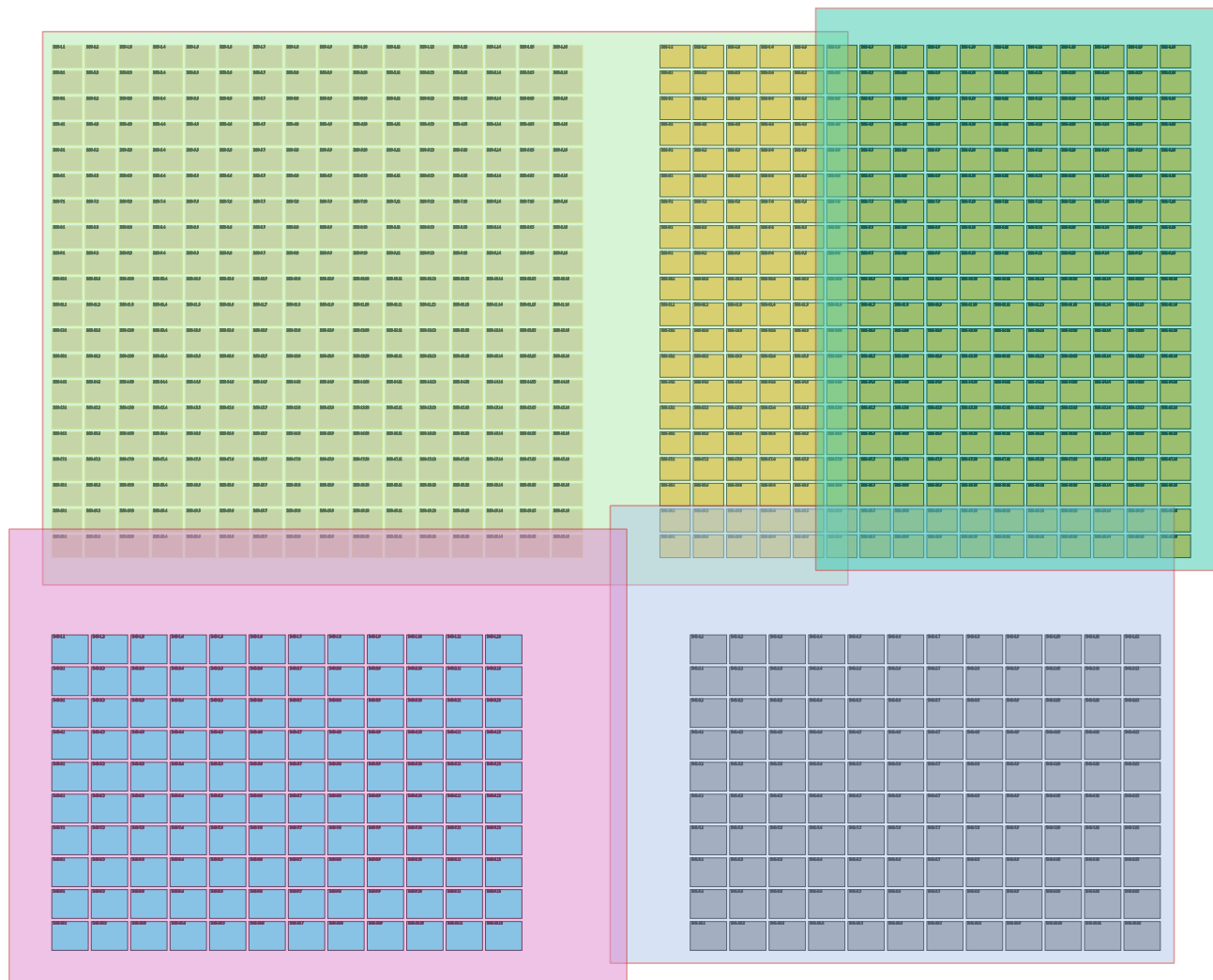


Figura 3: Exemplo de possível saída do comando `regs`.

A Figura 4 ilustra uma possível saída do comando `exp`. Note os segmentos vermelhos. Eles indicam os trechos das ruas que devem ser ampliados para melhorar o fluxo viário. Na prática, correspondem as arestas do grafo que tiveram a velocidade média aumentada.

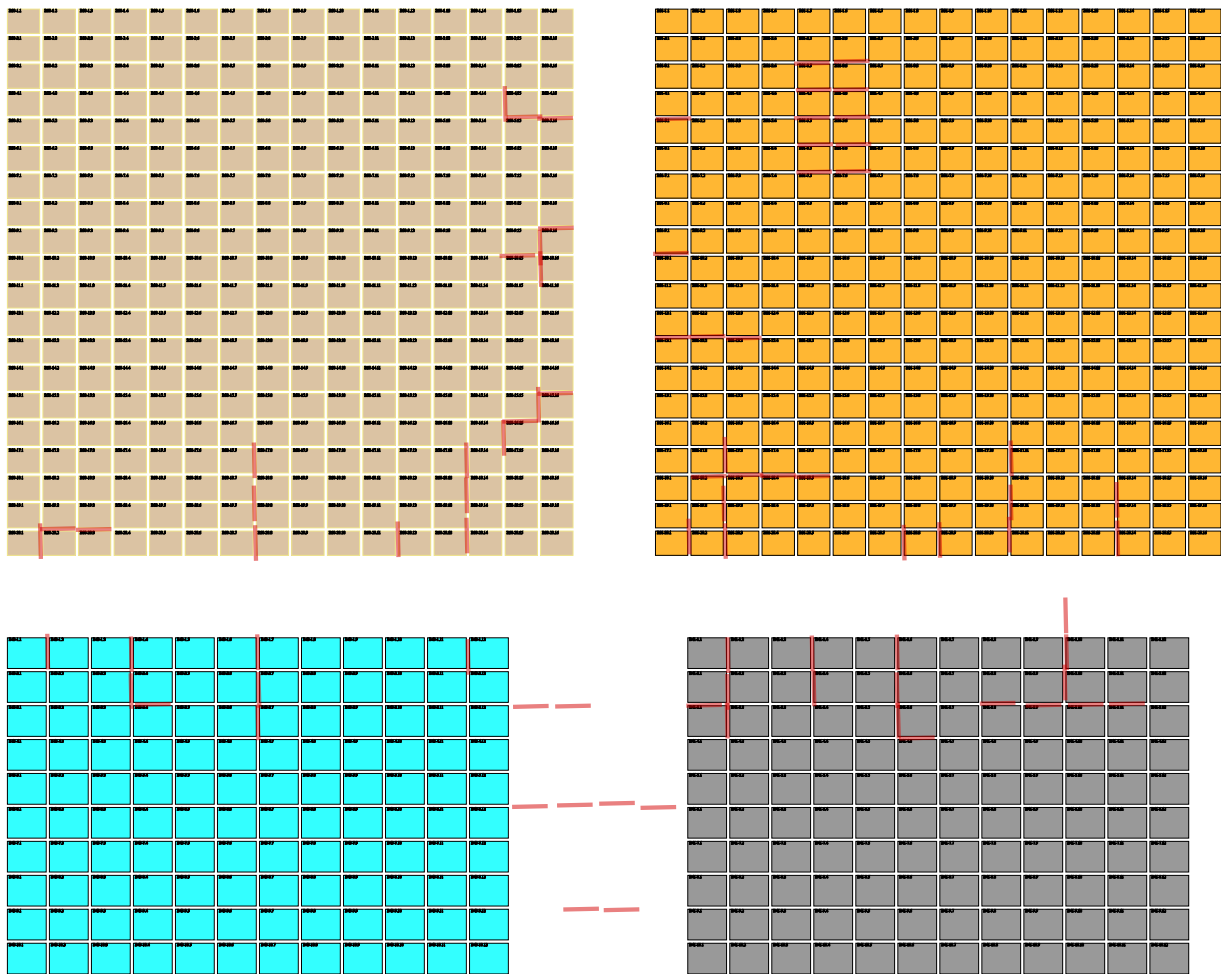


Figura 4: Exemplo de saída do comando `exp`.

A Implementação

As quadras devem ser armazenadas em alguma estrutura de dados que apresente um desempenho pelo menos razoável. Execuções demoradas (mais que 1 ou 2 minutos) serão interrompidas e consideradas incorretas.

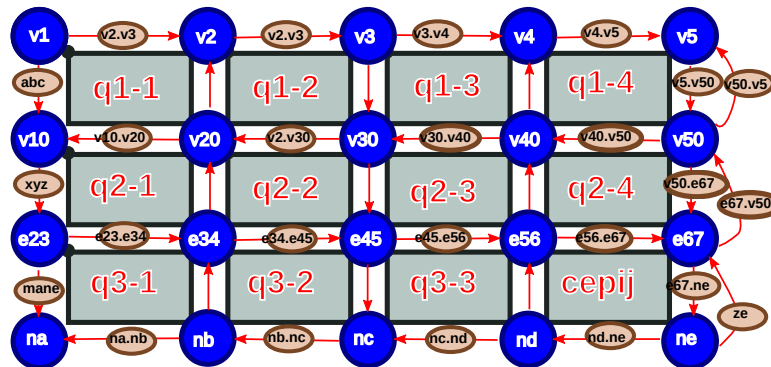
A implementação do grafo **deve** usar listas de adjacência.

Para a determinação de caminhos mínimos **deve** ser utilizado o algoritmo de **Dijkstra**.

As diversas estruturas de dados devem ser implementadas em módulos separados e bem documentados. Continua **expressamente proibido** definir `structs` em arquivos `.h`.

Para facilitar a implementação, alguns fatos podem ser considerados:

- Existem vértices do grafo posicionado no ponto médio das intersecções de ruas.
- Normalmente, existirão vértices no ponto médio de um segmento de rua. Isto é, o segmento de rua referente a uma face de uma quadra poderá ser, na prática, representado por duas (ou mais) arestas: (esquina1,meioquadra) e (meioquadra,esquina2). A figura abaixo ilustra onde serão posicionados os vértices extras (em lilás). Isto não altera o cálculo do percurso, apenas facilita posicionar o início e o fim do percurso.



Os percursos **devem** ser animados. Isto é, alguma figura (retângulo, círculo, etc)¹ deve transitar entre o início e o fim do percurso. A figura abaixo mostra um exemplo. Note que `<path/>` descreve o caminho a ser percorrido. Note o elemento `<animateMotion/>` dentro do elemento a ser animado (o círculo e o retângulo no nosso exemplo).

```
<path d="M10,110 A120,120 -45 0,1 110 10 A120,120 -45 0,1 10,110"
      stroke="lightgrey" stroke-width="2" fill="none" id="theMotionPath"/>

<path d="m 32.539299,36.987292 -1.233889,70.589218 50.211412,0.87769 0.458653,-26.239002
-28.795792,-0.503346 0.345472,-19.763936 60.819315,1.063112 -0.40189,22.991614 -13.58416,-0.23745
-0.36862,21.088178 49.3119,0.86196 1.16814,-66.828091 -54.976546,-0.960982" id="path10" />

<circle cx="" cy="" r="5" fill="red">
  <animateMotion dur="6s" repeatCount="indefinite">
    <mpath xlink:href="#theMotionPath"/>
  </animateMotion>
</circle>

<rect x="" y="" width="5" height="5" fill="blue">
  <animateMotion dur="6s" repeatCount="indefinite">
    <mpath xlink:href="#path10"/>
  </animateMotion>
</rect>
```

Para cada módulo, escrever um arquivo de teste unitário usando o módulo unity (conforme

¹ Para os valentes: desenhar um carro.

descrição geral. O makefile deve conter targets para compilação e execução dos testes unitários.

LEMBRE-SE: primeiro escrever o arquivo `.h`, commitar, escrever o arquivo de teste unitário, commitar, só depois escrever o arquivo `.c`, commitar.

IMPORTANTÍSSIMO: Sempre que resolver um bug, fazer commit esclarecendo qual era o bug, a causa e a solução. A mensagem do commit deve ser clara e objetiva.

LEMBRE-SE: o aviso imediatamente acima é **MUITÍSSIMO IMPORTANTÍSSIMO!!!**

A Saída

O programa deverá produzir um arquivo `.svg` e um arquivo `.txt` ambos com o mesmo nome base do arquivo `.geo`, como descrito anteriormente.

A Avaliação

Espera-se uma atitude pró-ativa para a aquisição dos conhecimentos (i.e., estudo) para resolver o problema proposto.

A avaliação consistirá da execução dos testes, da inspeção de código e da análise dos commits no repositório.

Qualquer **fraude** acarretará nota **NULA** a todos os envolvidos! É **expressamente proibida** a utilização de ferramentas de IA para a produção de código.

Critério	Desconto
Definir struct em arquivo <code>.h</code>	2.5
Modularização Pobre: <code>.h</code> mal projetado, mal documentado	até 2.0
Não implementado conforme especificado não implementar estruturas pedidas (gravíssimo)	tipicamente até 4.0, mas em casos graves, o desconto pode ser total
Procedimentos extensos e/ou complicados	até 1.0
Escolha/uso de estrutura pouco eficiente	até 2.0
Poucos commits, commits concentrados perto do final do prazo, commits sem mensagem elucidativa e clara. IMPORTANTÍSSIMO: fazer um commit para cada bug corrigido. Explicar na mensagem do commit claramente: (a) qual foi o bug; (b) como o bug foi corrigido. OBS.: também é esperado 1 commit por aula prática.	tipicamente até 3.0; porém, o padrão de commit pode indicar possível fraude .
Implementação ineficiente e/ou desidiosa.	Até 1.5.
Não usar o makefile provido	Em geral, nenhum desconto. Mas, se ocorrer problema de compilação ou execução que poderia ter sido evitado pelo uso do modelo do makefile provido, a consequência pode ser grave.

Erro de compilação	Nenhum teste será executado. Portanto, nota Zero, especialmente se for causado por negligência do aluno.
--------------------	---

O Programa

O nome do programa deve ser **ted** e aceitar até cinco parâmetros:

```
ted [-e path] -f arq.geo [-q consulta.qry] [-v arqvias.via] -o dir
```

O primeiro parâmetro (**-e**) indica o diretório base de entrada. É opcional. Caso não seja informado, o diretório de entrada é o diretório corrente da aplicação. O segundo parâmetro (**-f**) especifica o nome do arquivo de entrada que deve ser encontrado sob o diretório informado pelo primeiro parâmetro. O terceiro parâmetro (**-q**) é um arquivo de consultas. O último parâmetro (**-o**) indica o diretório onde os arquivos de saída (***.svg** e ***.txt**) devem ser colocados. Note que o nome do arquivo pode ser precedido por um caminho relativo; **dir e path** é um caminho absoluto ou relativo (ao diretório corrente).

O Que Entregar

Os arquivos-fontes devem estar em um repositório GIT. Submeter ao Classroom um arquivo-texto (.txt) com uma linha, com o formato abaixo. A URL fornecida será usada para clonar o repositório (git clone)

apelido url-do-repositorio

Exemplo:

joseMane https://github.com/zemane/t2.git

RESUMO DOS PARÂMETROS DO PROGRAMA

Parâmetro / argumento	Opcional	Descrição
-e path	S	Diretório-base de entrada (BED)
-f arq.geo	N	Arquivo com a descrição da cidade. Este arquivo deve estar sob o diretório BED .
-o path	N	Diretório-base de saída (BSD)
-q arqcons.qry	S	Arquivo com consultas. Este arquivo deve estar sob o diretório BED .
-v arq.via	S	Arquivo de vias (para construção do grafo)

ATENÇÃO:

* os fontes devem ser compilados com a opção `-fstack-protector-all`.

* adotamos o padrão C99. Usar a opção `-std=c99`.